

Full Stack Development Summary

Covered Topics:

- What is OOPS?
- Necessity and Advantage of OOPS
- OOPS Designs with real-time examples.
- What is meant by class and object?
- Relation between a Class and an Object
- How to create class and object
- Exercise Programs

What is OOPS

(Object-Oriented Programming System)?

- OOPS stands for Object-Oriented Programming System
- It is a programming approach where we design programs using objects that represent real-world entities
- Each object represents a real-world entity that has attributes (data/variables) and behaviors (methods/functions).

Necessity and Advantage of OOPS

- Before OOPS, programming was done in Procedural Languages (like C) — where data and functions were separate.
- OOPS combines both data and behavior into one unit (object), making programs more modular, reusable, and secure.
- The primary advantages of using Object-Oriented Programming (OOPs) in Java include enhanced code reusability, superior data security, simplified debugging, and better software maintainability.

OOPS Designs with real-time examples

Real-Life Analogy:

Imagine a **School Management System**:

- **Classes:** Student, Teacher, Subject, School
- **Objects:** Specific students, teachers, subjects
- **Attributes:** name, age, subject, marks
- **Methods:** attendClass(), teach(), calculateMarks()

OOPs Concepts

1. Class
2. Objects
3. Inheritance
4. Encapsulation
5. Polymorphism
6. Abstraction

What is meant by class and object?

A class is a **blueprint** or template used to create individual entities, while an **object is a specific instance** of that class created in memory

Key Differences At a Glance

Feature 	Class	Object
Concept	A blueprint or definition.	A physical instance of the blueprint.
Memory	Does not allocate memory when declared.	Allocates memory when instantiated.
Existence	It is a logical entity only.	It is a physical entity.
Quantity	Declared only once per file/program.	Multiple objects can be created from one class.
Keyword	Created using the <code>class</code> keyword.	Created using the <code>new</code> keyword.

Relation between a Class and an Object

The relation between a class and an object is a **one-to-many relationship based on instantiation**. A class acts as the abstract definition, while an object acts as the concrete realization of that definition.

- The Blueprint and the Structure
- The Relationship Matrix
- Visual Relationship Flow

How to create class and object

```

class ClassName {
    // data members
    // methods
}

public class MainClass {
    public static void main(String[] args) {
        ClassName obj = new ClassName(); // object creation
        obj.methodName();                // calling method
    }
}

```

To create a class and object in Java, you use the **class keyword** to define the blueprint and the **new keyword** to instantiate an object from that blueprint.

Exercise Programs

Example Program

```

class Animal {
    String name;
    void sound() {
        System.out.println(name + " makes a sound!");
    }
}

public class AnimalMain {
    public static void main(String[] args) {
        Animal dog = new Animal();
        dog.name = "Dog";
        dog.sound();

        Animal cat = new Animal();
        cat.name = "Cat";
        cat.sound();
    }
}

```

```

Dog makes a sound!
Cat makes a sound!

```